

PRÁCTICA: IMU – Pedómetro y Reconstrucción de Trayectoria

Acelerómetro MPU-6886 + Giróscopo | M5StickC Plus 2
M5StickC Plus 2 | Python (análisis offline en Windows)
APS-GRIA – Universidad de Vigo – Curso 2025-2026

Introducción

En esta práctica se trabaja con datos reales capturados por la IMU (Inertial Measurement Unit) del M5StickC Plus 2, que integra un acelerómetro y un giróscopo de 3 ejes (MPU-6886). Los datos han sido grabados mientras una persona camina, gira y corre por los pasillos del edificio.

A partir de este fichero CSV de datos brutos, el alumno debe implementar en Python sobre Windows un pipeline completo de procesamiento de señal inercial que permita:

- **Detectar pasos** a partir de picos en la magnitud de la aceleración.
- **Detectar giros** integrando la velocidad angular del giróscopo (eje Z).
- **Estimar distancias** con longitud de paso fija y con longitud variable proporcional a la intensidad del paso.
- **Reconstruir la trayectoria 2D** combinando pasos y orientación para generar un mapa del recorrido.

No se programa el M5: todo el análisis se realiza offline en Python con matplotlib, numpy y pandas.

Material

- PC con Python 3.10+ (Windows)
- Librerías: numpy, pandas, matplotlib, scipy (signal)
- Fichero de datos: `imu_log2.csv`

El fichero contiene 14.101 muestras a ~172 Hz durante ~82 segundos. Columnas: `ts_ms` (timestamp en ms), `ax/ay/az` (aceleración en g), `gx/gy/gz` (velocidad angular en °/s). El dispositivo se llevaba en la mano con el eje Z apuntando aproximadamente hacia arriba.

Formato de los datos

Columna	Unidad	Descripción
---------	--------	-------------

`ts_ms` – Timestamp en milisegundos (reloj del ESP32, monotónico).

`ax`, `ay`, `az` – Aceleración en cada eje, en unidades de g ($1\text{ g} \approx 9.81\text{ m/s}^2$). En reposo con el dispositivo horizontal: `az` $\approx 1.0\text{ g}$.

`gx`, `gy`, `gz` – Velocidad angular en °/s. `gz` corresponde al giro en el plano horizontal (yaw).

Hay muestras duplicadas (mismo valor, timestamps consecutivos) porque el M5 lee la IMU más rápido de lo que el sensor actualiza su registro interno. El Bloque 1 incluye la eliminación de duplicados.

Bloque 1 – Carga, limpieza y visualización (25 min)

Fichero de partida: `imu_log2.csv`

Objetivo

Cargar los datos, eliminar duplicados, calcular la magnitud de la aceleración y visualizar las señales para entender su estructura antes de procesarlas.

Tareas a implementar

Tarea 1 – Carga y limpieza

Carga el CSV con pandas. Convierte `ts_ms` a segundos relativos (restando el primer timestamp y dividiendo por 1000). Elimina filas duplicadas consecutivas donde todos los valores de sensor sean idénticos (usa `df.drop_duplicates(subset=['ax', 'ay', 'az', 'gx', 'gy', 'gz'])`). Cuenta cuántas muestras se eliminaron.

Tarea 2 – Magnitud de la aceleración

Calcula la magnitud del vector de aceleración:

$$|a| = \sqrt{ax^2 + ay^2 + az^2}$$

En reposo, $|a| \approx 1.0$ g. Al caminar, los impactos del pie provocan picos por encima y valles por debajo de 1 g.

Tarea 3 – Gráficas exploratorias

Genera una figura con 3 subplots verticales (compartiendo eje X de tiempo):

- **Subplot 1:** $|a|$ vs tiempo. Añade una línea horizontal en 1.0 g.
- **Subplot 2:** gz (velocidad angular eje Z) vs tiempo.
- **Subplot 3:** ax, ay, az individuales vs tiempo.

Pista: usa `plt.subplots(3, 1, sharex=True, figsize=(14, 8))` y ajusta con `plt.tight_layout()`.

Experimentos

Exp. A – Identificación visual de patrones

Observa las gráficas y contesta en comentarios del código:

Pregunta	Respuesta
¿Se distinguen visualmente los pasos como picos en $ a $?	
¿Se identifican los giros como ráfagas en gz?	
¿Hay zonas con mayor frecuencia de picos (corriendo vs andando)?	
¿Cuántas muestras quedaron tras eliminar duplicados?	

Bloque 2 – Detección de pasos (30 min)

Objetivo

Implementar un detector de pasos basado en la detección de picos en la magnitud de la aceleración, con filtrado previo para eliminar ruido de alta frecuencia.

Marco teórico

Al caminar, cada paso genera un impacto que produce un pico característico en $|a|$. La cadencia normal de marcha es de 1.5–2.0 pasos/segundo, lo que implica un intervalo mínimo de ~400–600 ms entre pasos. Al correr, la cadencia sube a 2.5–3.0 pasos/s y la amplitud de los picos es mucho mayor.

El algoritmo básico es:

1. 1. Filtrar $|a|$ con un filtro paso bajo para eliminar vibraciones de alta frecuencia.
2. 2. Detectar picos que superen un umbral sobre la línea base de 1.0 g.
3. 3. Aplicar un intervalo mínimo entre picos para evitar doble conteo.

scipy.signal.find_peaks permite especificar *height* (umbral), *distance* (intervalo mínimo en muestras) y *prominence* (prominencia mínima del pico respecto a su entorno).

Tareas a implementar

Tarea 1 – Filtrado paso bajo

Aplica un filtro Butterworth de orden 4, frecuencia de corte 5 Hz, sobre la señal $|a|$. Para calcular la frecuencia normalizada necesitas la frecuencia de muestreo real: calcula $fs = len(df) / duracion_total_s$.

Usa `scipy.signal.butter(4, 5, btype='low', fs=fs)` para obtener los coeficientes y `scipy.signal.filtfilt(b, a, mag)` para aplicar el filtro sin desfase.

Tarea 2 – Detección de picos

Usa `scipy.signal.find_peaks()` sobre la señal filtrada con los siguientes parámetros iniciales:

- `height=1.05` (umbral sobre 1 g)
- `distance=int(fs*0.4)` (400 ms mínimo entre pasos)
- `prominence=0.05` (prominencia mínima)

Guarda los índices de los picos detectados y sus propiedades (altura, prominencia).

Tarea 3 – Visualización de la detección

Sobre la gráfica de $|a|$ filtrada, marca cada paso detectado con un punto rojo. Añade un título indicando el número total de pasos y la cadencia media (pasos/s).

Experimentos

Exp. A – Ajuste del umbral

Prueba distintos valores del parámetro `height` y anota cómo cambia el conteo:

height	Pasos detectados	Cadencia (pasos/s)	Observaciones

--	--	--	--

Exp. B – Efecto de la frecuencia de corte

Cambia la frecuencia de corte del filtro Butterworth y observa el efecto:

f_corte (Hz)	Pasos detectados	Forma de la señal	Observaciones

Frecuencias demasiado bajas suavizan tanto que los picos desaparecen. Frecuencias demasiado altas dejan ruido que genera falsos picos.

Exp. C – Segmentación caminar vs correr

Observando la amplitud de los picos, ¿es posible distinguir automáticamente las zonas de carrera de las de marcha? Propone un umbral de amplitud que separe ambos regímenes y cuenta los pasos en cada modo.

Segmento	t_inicio (s)	t_fin (s)	Nº pasos	Amplitud media pico (g)	Modo

Bloque 3 – Detección de giros y estimación de heading (25 min)

Objetivo

Integrar la velocidad angular del giróscopo para obtener la orientación (heading) del usuario a lo largo del recorrido, y detectar los eventos de giro significativos.

Marco teórico

El giróscopo mide la velocidad angular instantánea en °/s. Para obtener el ángulo acumulado (heading), se integra numéricamente:

$$\text{heading}(t) = \text{heading}(t-1) + g_z(t) \times dt$$

donde dt es el intervalo de tiempo entre muestras consecutivas. Esta integración acumula error (drift) con el tiempo debido al bias del giróscopo, pero para recorridos cortos (<2 minutos) el error es manejable.

Un giro significativo se define como un cambio de heading de más de 30° en menos de 3 segundos. Los giros típicos en un pasillo son de ~90°.

Tareas a implementar

Tarea 1 – Compensación de bias

Antes de integrar, calcula el bias del giróscopo: toma la media de g_z durante los primeros 0.5 segundos (asumiendo que el usuario está quieto al inicio). Resta este bias a toda la señal g_z .

$$g_{z_corregido} = g_z - \text{bias}$$

El bias típico del MPU-6886 es de 1–3 °/s. Sin compensarlo, tras 80 segundos el heading acumularía ~80–240° de error.

Tarea 2 – Integración del heading

Integra $g_{z_corregido}$ usando integración trapezoidal (o simple Euler) para obtener el heading acumulado en grados. Aplica previamente un filtro paso bajo a g_z (misma técnica que en el Bloque 2, $f_{\text{corte}} = 2$ Hz) para reducir el ruido antes de integrar.

Tarea 3 – Detección de giros

Detecta los giros analizando la derivada del heading o los segmentos donde $|g_{z_corregido}|$ supera un umbral (p.ej. 20 °/s) durante más de 200 ms. Para cada giro, registra: instante, duración, y ángulo total girado ($\Delta\text{heading}$).

Tarea 4 – Visualización

Genera una gráfica con 2 subplots:

- **Subplot 1:** g_z original y g_z filtrado vs tiempo.
- **Subplot 2:** heading acumulado vs tiempo, con marcas verticales en cada giro detectado y anotación del ángulo girado.

Experimentos

Exp. A – Inventario de giros

Rellena la tabla con los giros detectados:

Giro #	t (s)	Duración (s)	$\Delta\text{heading}$ (°)	Dirección (I/D)

Exp. B – Efecto del bias

Compara el heading final con y sin compensación de bias:

Condición	Heading final (°)	Drift estimado (°)

Pregunta: si el recorrido es un circuito cerrado, el heading final debería ser $\sim 0^\circ$ (o múltiplo de 360°). ¿Cuánto drift residual queda tras la compensación de bias?

Bloque 4 – Reconstrucción de trayectoria 2D (40 min)

Objetivo

Combinar la detección de pasos con la orientación estimada para reconstruir la trayectoria recorrida en un plano 2D, utilizando dos modelos de longitud de paso.

Marco teórico

La técnica se llama Pedestrian Dead Reckoning (PDR). En cada paso detectado se avanza una distancia d en la dirección del heading actual:

$$x(n) = x(n-1) + d(n) \times \cos(\text{heading}(n))$$

$$y(n) = y(n-1) + d(n) \times \sin(\text{heading}(n))$$

Para la longitud del paso $d(n)$ se usan dos modelos:

- **Modelo A – Paso fijo:** $d = 0.70$ m (constante, valor típico para un adulto).
- **Modelo B – Paso proporcional a la intensidad:** la amplitud del pico de aceleración se correlaciona con la fuerza de la pisada. Un modelo simple es:

$$d(n) = K \times (\text{pico}(n) - 1.0)^{(1/4)}$$

donde K es un factor de escala (típicamente 0.5–0.8) y $\text{pico}(n)$ es la magnitud del pico de aceleración del paso n . La raíz cuarta comprime el rango dinámico para que los pasos de carrera no sean desproporcionadamente largos.

El modelo B es una aproximación empírica. Weinberg (2002) propuso $d = K \times (a_{\text{max}} - a_{\text{min}})^{(1/4)}$ como estimador robusto de longitud de paso. Aquí usamos la variante simplificada con el pico sobre 1 g.

Tareas a implementar

Tarea 1 – Heading en cada paso

Para cada paso detectado en el Bloque 2, obtiene el heading correspondiente del Bloque 3 (interpolando al instante del pico). Almacena un array de tuplas: (t_paso, heading_paso, amplitud_pico).

Tarea 2 – Trayectoria con paso fijo (Modelo A)

Implementa el PDR con $d = 0.70$ m. Genera un array de posiciones (x, y) a partir de la posición inicial (0, 0).

Tarea 3 – Trayectoria con paso variable (Modelo B)

Implementa el PDR con $d(n) = K \times (\text{pico}(n) - 1.0)^{0.25}$. Prueba con $K = 0.6$ como valor inicial.

Tarea 4 – Visualización comparativa

Genera una figura con:

- **Plot principal:** ambas trayectorias (A y B) superpuestas en un gráfico X-Y con ejes iguales (`plt.axis('equal')`). Marca el punto de inicio con un círculo verde y el final con un cuadrado rojo.
- **Codificación de color:** colorea los segmentos según la velocidad (pasos/s local, usando una ventana de 5 pasos). Usa un colormap como `plt.cm.RdYlGn_r` (rojo = rápido/corriendo, verde = lento/andando).
- **Anotación de giros:** en los puntos donde se detectaron giros (Bloque 3), añade flechas indicando la dirección del giro.

Experimentos

Exp. A – Comparación de modelos

Métrica	Modelo A (fijo)	Modelo B (variable)
---------	-----------------	---------------------

Rellena con:

- Distancia total recorrida (m)
- Longitud media de paso (m)
- Desviación estándar de longitud de paso (m)
- Error de cierre (distancia entre punto final e inicio, si el recorrido es cerrado)

Exp. B – Calibración de K

Si conoces la distancia real de un tramo recto del recorrido, ajusta K para que el modelo B la reproduzca:

K	Distancia total (m)	Paso medio (m)	Error de cierre (m)

Exp. C – Análisis de la trayectoria

Observa la trayectoria generada y responde:

¿Se reconoce la geometría del pasillo? ¿Los giros de 90° se aprecian en el mapa?

¿Dónde divergen más los dos modelos: en tramos rectos o en giros?

¿El error de cierre es menor con paso fijo o variable? ¿Por qué?

Preguntas de Reflexión

Responde al menos 3 de las siguientes preguntas en comentarios del código o en un informe breve adjunto.

#	Pregunta
R1	El drift del giróscopo crece linealmente con el tiempo. Si el recorrido durase 10 minutos en vez de 80 segundos, ¿cuánto heading falso acumularía con el bias que mediste? ¿Qué sensor adicional podría corregirlo? Investiga brevemente qué es un filtro complementario o un filtro de Kalman en este contexto.
R2	El modelo de paso variable usa la raíz cuarta de la amplitud. ¿Por qué no se usa directamente la amplitud? Piensa en qué pasaría con pasos de carrera (pico ~1.8 g) vs marcha (pico ~1.2 g): ¿qué relación de longitudes daría la amplitud directa vs la raíz cuarta?
R3	El PDR acumula error con cada paso: el error de posición crece como $\sqrt{N}$ donde N es el número de pasos (random walk). ¿Cómo podrías corregir este error si tienes puntos de referencia conocidos en el edificio (p.ej. puertas o esquinas)?
R4	En esta práctica asumimos que el dispositivo se lleva con orientación fija. Si el usuario lleva el M5 en el bolsillo y este rota libremente, ¿qué cambiaría en los algoritmos? ¿Seguiría funcionando la detección de pasos? ¿Y la detección de heading?
R5	Compara las ventajas y limitaciones de este enfoque inercial (IMU) frente a otros sistemas de posicionamiento interior: WiFi fingerprinting, BLE beacons, UWB. ¿En qué escenarios la IMU sería la mejor opción?
R6	El acelerómetro mide gravedad + aceleración lineal. ¿Cómo podrías usar la componente de gravedad para detectar si el usuario sube o baja escaleras? ¿Qué eje cambiaría y cuánto esperarías que cambiase la componente az durante una subida?